

DATA STRUCTURES AND ALGORITHMS (LAB)

Assignment 01 Part 01



By

MISBAH KHALID

FA25-BCS-005

Submitted to,

Mr. BILAL BUKHARI

September 16, 2026

Department of Computer Sciences
COMSATS University Islamabad,
Attock Campus

CODE:

```
#include <iostream>
#include <cmath>
using namespace std;

const int CAPACITY = 20;

struct ArrayList
{
    int data[CAPACITY];
    int size = 0;
};

// Insert at end
bool insertEnd(ArrayList &list, int value)
{
    if (list.size == CAPACITY)
        return false;

    list.data[list.size] = value;
    list.size++;

    return true;
}

// Insert at beginning
bool insertAtBeginning(ArrayList &list, int value)
{
    if (list.size == CAPACITY)
        return false;

    // Shift elements to the right
    for (int i = list.size; i > 0; i--)
    {
        list.data[i] = list.data[i - 1];
    }

    list.data[0] = value;
    list.size++;

    return true;
}

// Delete element at a position
bool deleteAtPosition(ArrayList &list, int position)
{

```

```

    if (position < 0 || position >= list.size)
        return false;

    // Shift elements to the left
    for (int i = position; i < list.size - 1; i++)
    {
        list.data[i] = list.data[i + 1];
    }

    list.size--;

    return true;
}

// Display ArrayList
void displayList(const ArrayList &list)
{
    for (int i = 0; i < list.size; i++)
    {
        cout << list.data[i] << " ";
    }
    cout << endl;
}

int main()
{
    ArrayList list;

    int *ptr = list.data;
    int *minPtr = list.data;
    int *maxPtr = list.data;
    int *medianPtr = list.data;
    int *closestPtr = list.data;

    int sum = 0;
    int closestPosition;

    double generalAverage;
    double specialAverage;
    double averageDifference;
    double finalScore;

    insertEnd(list, 18);
    insertEnd(list, 7);
    insertEnd(list, 45);
    insertEnd(list, 11);
    insertEnd(list, 36);
    insertEnd(list, 05);

```

```

insertEnd(list, 21);
insertEnd(list, 13);
insertEnd(list, 29);

cout << "Initial ArrayList: ";
displayList(list);

ptr = list.data;
minPtr = list.data;
maxPtr = list.data;
sum = 0;

for (int i = 0; i < list.size; i++)
{
    sum = sum + *ptr;

    if (*ptr < *minPtr)
        minPtr = ptr;

    if (*ptr > *maxPtr)
        maxPtr = ptr;

    ptr++;
}

cout << "Minimum Value: " << *minPtr << endl;
cout << "Maximum Value: " << *maxPtr << endl;
cout << "Sum: " << sum << endl;

int temp[CAPACITY];

// Copy logical elements into temporary array
for (int i = 0; i < list.size; i++)
{
    temp[i] = list.data[i];
}

// Manual Bubble Sort
for (int i = 0; i < list.size - 1; i++)
{
    for (int j = 0; j < list.size - i - 1; j++)
    {
        if (temp[j] > temp[j + 1])
        {
            int t = temp[j];
            temp[j] = temp[j + 1];
            temp[j + 1] = t;
        }
    }
}

```

```

    }
}

// Since there are 9 elements
int medianValue = temp[list.size / 2];

// Find median in original ArrayList
medianPtr = list.data;

for (int i = 0; i < list.size; i++)
{
    if (*medianPtr == medianValue)
        break;

    medianPtr++;
}

cout << "Median Value: " << *medianPtr << endl;

generalAverage = (double)sum / list.size;

specialAverage =
    (*minPtr + *medianPtr + *maxPtr) / 3.0;

// Find closest value
ptr = list.data;
closestPtr = list.data;

double smallestDifference =
    fabs(*ptr - specialAverage);

for (int i = 0; i < list.size; i++)
{
    double difference =
        fabs(*ptr - specialAverage);

    if (difference < smallestDifference)
    {
        smallestDifference = difference;
        closestPtr = ptr;
    }

    ptr++;
}

// Pointer arithmetic
closestPosition = closestPtr - list.data;

cout << "General Average: " << generalAverage << endl;

```

```

cout << "Special Average: " << specialAverage << endl;
cout << "Closest Value: " << *closestPtr << endl;
cout << "Position of Closest Value: "
    << closestPosition << endl;

averageDifference =
    fabs(generalAverage - specialAverage);

finalScore =

    fabs(*closestPtr - generalAverage)
    + fabs(*closestPtr - specialAverage)
    + averageDifference;

cout << "Difference Between Averages: "
    << averageDifference << endl;

cout << "Final Score: "
    << finalScore << endl;

// Store position before deletion
closestPosition = closestPtr - list.data;

// Delete closest element
deleteAtPosition(list, closestPosition);

cout << "ArrayList After Deletion: ";
displayList(list);

// Round special average
int roundedSpecialAverage =
    (int)round(specialAverage);

// Insert rounded value at beginning
insertAtBeginning(list, roundedSpecialAverage);

cout << "Final ArrayList After Insertion: ";
displayList(list);

return 0;
}

```

OUTPUT:

```
227
228
229
230 Initial ArrayList: 18 7 45 11 36 5 21 13 29
231 Minimum Value: 5
232 Maximum Value: 45
233 Sum: 185
234 Median Value: 18
235 General Average: 20.5556
236 Special Average: 22.6667
237 Closest Value: 21
238 Position of Closest Value: 6
239 Difference Between Averages: 2.11111
240 Final Score: 4.22222
241 ArrayList After Deletion: 18 7 45 11 36 5 13 29
242 Final ArrayList After Insertion: 23 18 7 45 11 36 5 13 29
243
244 -----
245 Process exited after 0.4299 seconds with return value 0
246 Press any key to continue . . . |
247
248
249
250
```